VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

NØSEE

Software Architecture Description – C4 Model

Autor:
Daniel Stiven Perez Cordoba
Cristhian Eduardo Osorio Restrepo

Versión: 1.0
Fecha: 11/2/2026

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

1. RESUMEN ARQUITECTÓNICO

NØSEE es una plataforma colaborativa web de comparación de precios orientada al consumidor colombiano. A diferencia de los comparadores de precios tradicionales, delega la recopilación y validación de la información en la misma comunidad de usuarios, combinando inteligencia colectiva con algoritmos de optimización para ayudar a los compradores a tomar mejores decisiones con el menor costo y tiempo posible.

1.1 Usuarios principales

La plataforma NØSEE está diseñada para atender cuatro perfiles de usuario diferenciados, cada uno con interacciones y responsabilidades específicas dentro del ecosistema:

- **Usuario de la Comunidad:** Consumidor que busca ahorrar dinero al comparar precios. Puede actuar como colaborador publicando precios o como consultor.

- **Moderador de Contenido**: Usuario con privilegios elevados responsable de la calidad y veracidad de la información publicada en la plataforma.

- **Administrador del Sistema**: Rol técnico-gerencial encargado de la gestión global de la plataforma, configuraciones y mantenimiento de usuarios con privilegios.

- **Repartidor**: Usuario que ofrece el servicio de compra y entrega a domicilio a partir de los pedidos generados por otros usuarios.

Adicionalmente, el sistema interactúa con un actor externo sistémico:

Servicio de Geolocalización (API Externa): Proveedor de mapas y geocodificación (Google Maps API) que permite ubicar tiendas físicas, calcular distancias y trazar rutas óptimas entre puntos de interés.

1.2 Problema que resuelve

Los consumidores colombianos enfrentan una asimetría de información significativa al momento de realizar sus compras: deben visitar físicamente múltiples tiendas o consultar varios sitios web de forma independiente para comparar precios, sin garantía de encontrar la mejor opción. Esta situación genera tres problemáticas centrales:

- Dispersión de la información: No existe una fuente centralizada, confiable y actualizada que consolide precios de comercio físico y digital en un solo lugar.

- Ineficiencia económica: Los consumidores compran sin conocer el precio real del mercado, perdiendo oportunidades de ahorro que podrían lograrse al distribuir la compra entre diferentes establecimientos.

- Ineficiencia temporal: La búsqueda manual de precios consume tiempo significativo sin garantizar resultados óptimos.

NØSEE resuelve estos problemas mediante tres pilares fundamentales:

- Democratización de la información: La comunidad de usuarios contribuye publicando precios reales de productos en tiendas físicas y virtuales, creando una base de datos dinámica validada por pares.

- Optimización de recursos: El algoritmo de «Cesta Óptima» calcula automáticamente qué productos conviene comprar en qué tienda, minimizando el costo total y considerando la ubicación geográfica del usuario.

- Confianza comunitaria: El sistema de votación (upvote/downvote), la moderación controlada y el algoritmo de reputación filtran información errónea o desactualizada, garantizando la calidad de los datos.

1.3 Alcance

El alcance del sistema NØSEE cubre las siguientes capacidades funcionales:

Incluido en el sistema:

- Registro y autenticación de usuarios con control de roles (Usuario Básico, Usuario Verificado, Moderador, Administrador, Repartidor).

- Publicación colaborativa de precios de productos con geolocalización de tiendas físicas y virtuales.

- Sistema de votación y validación comunitaria (upvote/downvote) con algoritmo de reputación y puntos.

- Motor de búsqueda y filtrado avanzado por ubicación, precio y tipo de tienda.

- Módulo de gestión de pedidos con algoritmo de optimización de cesta (programación lineal o greedy).

- Servicio de domicilio con seguimiento GPS en tiempo real del repartidor.

- Procesamiento de pagos mediante integración con servicio externo.

- Módulo de reportes para contenido inapropiado o precios incorrectos.

- Módulo de reportes para contenido inapropiado o precios incorrectos.
- Sistema de gamificación con puntos y niveles de reputación.
- Historial de precios y análisis de tendencias.

Excluido del sistema (fuera de alcance):

- Procesamiento directo de pagos (se delega a servicio externo).
- Gestión de inventarios de tiendas comerciales.
- Sistema propio de logística o gestión de envíos.
- Integración automática con APIs propietarias de tiendas (fase inicial).

1.4 Procesos involucrados en el sistema NØSEE

La operación de NØSEE se estructura en cuatro procesos de negocio interrelacionados y dependientes entre sí:

Proceso 1 - Gestión de Usuario y Autenticación
Administra el ciclo de vida completo de los usuarios: registro, autenticación, recuperación de contraseñas, gestión de perfiles y asignación de roles. Implementa el sistema de reputación que incentiva la participación de calidad. Es el proceso base del que dependen todos los demás, ya que el usuario debe estar autenticado y en el rol correcto para acceder a cualquier funcionalidad.

Proceso 2 - Gestión de Publicaciones de Precios
Permite a los usuarios registrar precios de productos en tiendas físicas y virtuales con evidencia fotográfica y geolocalización. Incorpora validación comunitaria mediante votaciones, detección de duplicados, cálculo de puntuación de confiabilidad y moderación de contenido. Es la fuente de datos principal del sistema: provee los precios que alimentan el algoritmo de optimización del Proceso 3.

Proceso 3 - Gestión de Pedido y Optimización de Compra
Convierte la lista de compra del usuario en un pedido formal, ejecutando el algoritmo de cesta óptima para calcular la combinación de tiendas que minimice el costo total dentro de un radio geográfico definido. Genera la ruta óptima entre tiendas, muestra el ahorro estimado y deja el pedido disponible para ser atendido por un repartidor (Proceso 4) o gestionado por el propio usuario.

Proceso 4 - Servicio de Domicilio con Pago Externo
Gestiona el servicio de entrega a domicilio de principio a fin: asignación del repartidor disponible, seguimiento GPS en tiempo real durante la compra en tiendas y el trayecto de entrega, gestión de novedades (productos no disponibles), procesamiento del pago mediante servicio externo y actualización de la reputación del repartidor. Retroalimenta al Proceso 2 cuando detecta precios desactualizados.

Dependencias entre procesos:

P1 → P2, P3, P4: Todo usuario debe estar autenticado con el rol correcto para interactuar con cualquier proceso.

P2 → P3: Los precios publicados y validados en P2 son la base de datos que consume el algoritmo de optimización de P3.

P3 → P4: El pedido optimizado y confirmado en P3 queda disponible para ser tomado por un repartidor en P4.

P4 → P2: Las novedades de productos no encontrados reportadas en P4 pueden activar alertas de desactualización en P2.

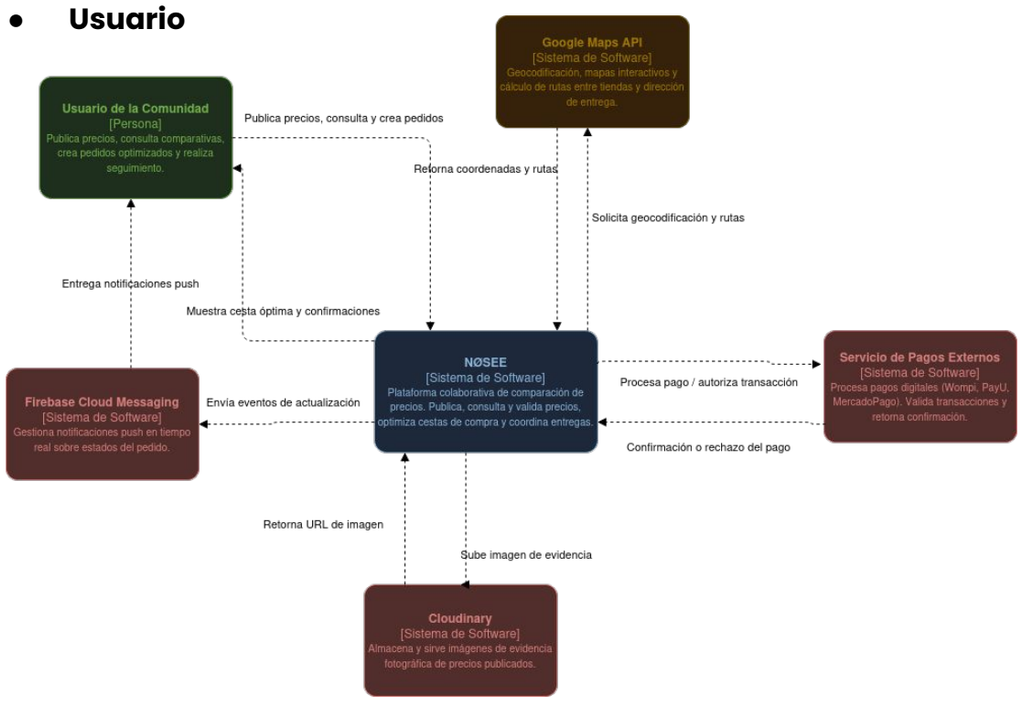
2. C4MODEL

Aunque NØSEE contempla los roles de Usuario de la Comunidad, Moderador, Administrador y Repartidor, las vistas C4 presentadas en este documento se enfocan en el flujo principal del usuario consumidor: desde la consulta y publicación de precios hasta la generación del pedido optimizado y su entrega a domicilio (app móvil, backend, base de datos e integraciones externas). Las capacidades del Moderador, el Administrador y el Repartidor forman parte del sistema global descrito en el Ítem 1, pero sus vistas detalladas se documentarán en futuras versiones de este documento

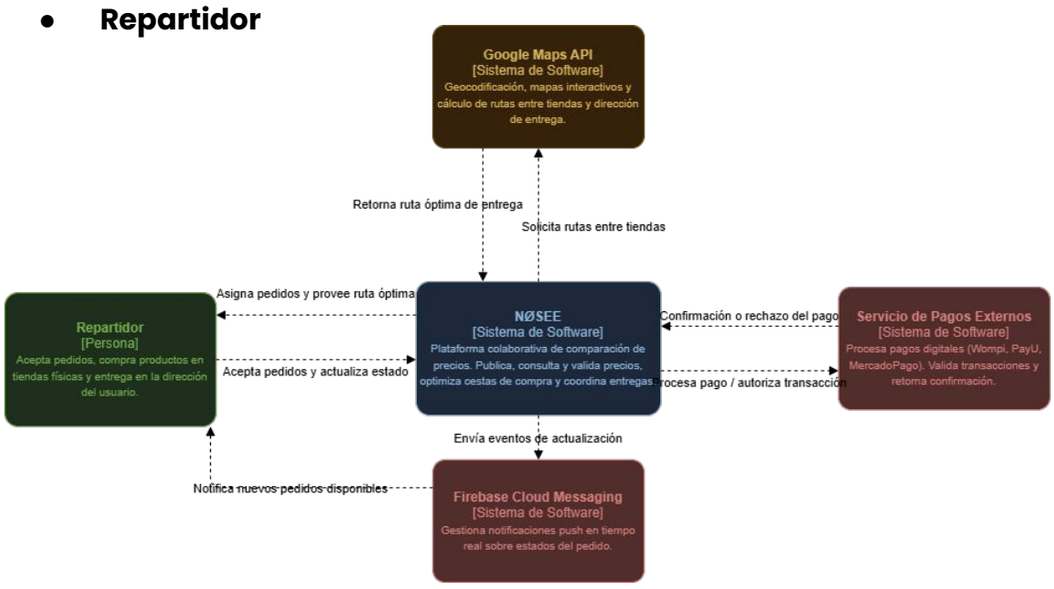
2.1 Diagrama de contexto (N1)

Presenta el sistema NØSEE como "caja negra", usuarios y sistemas externos.

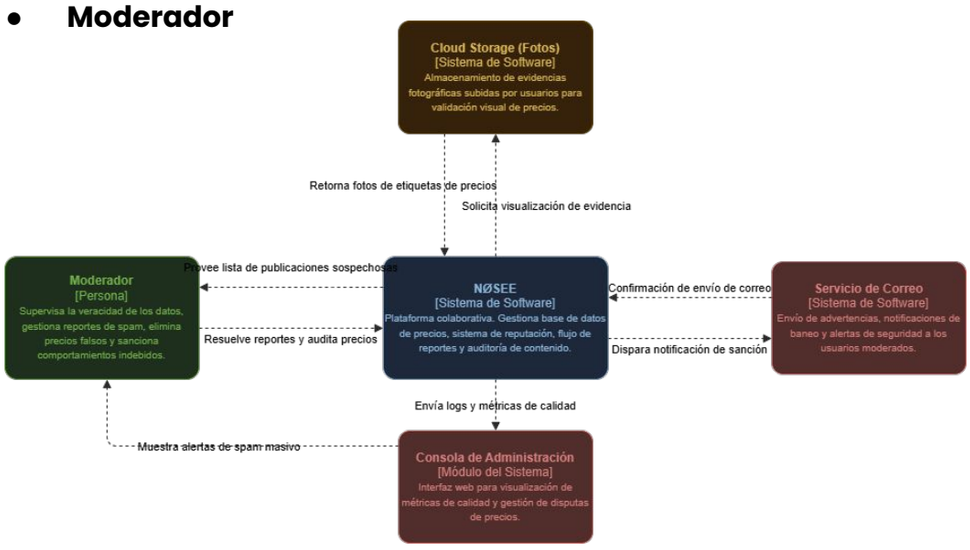
- **Usuario**



- **Repartidor**

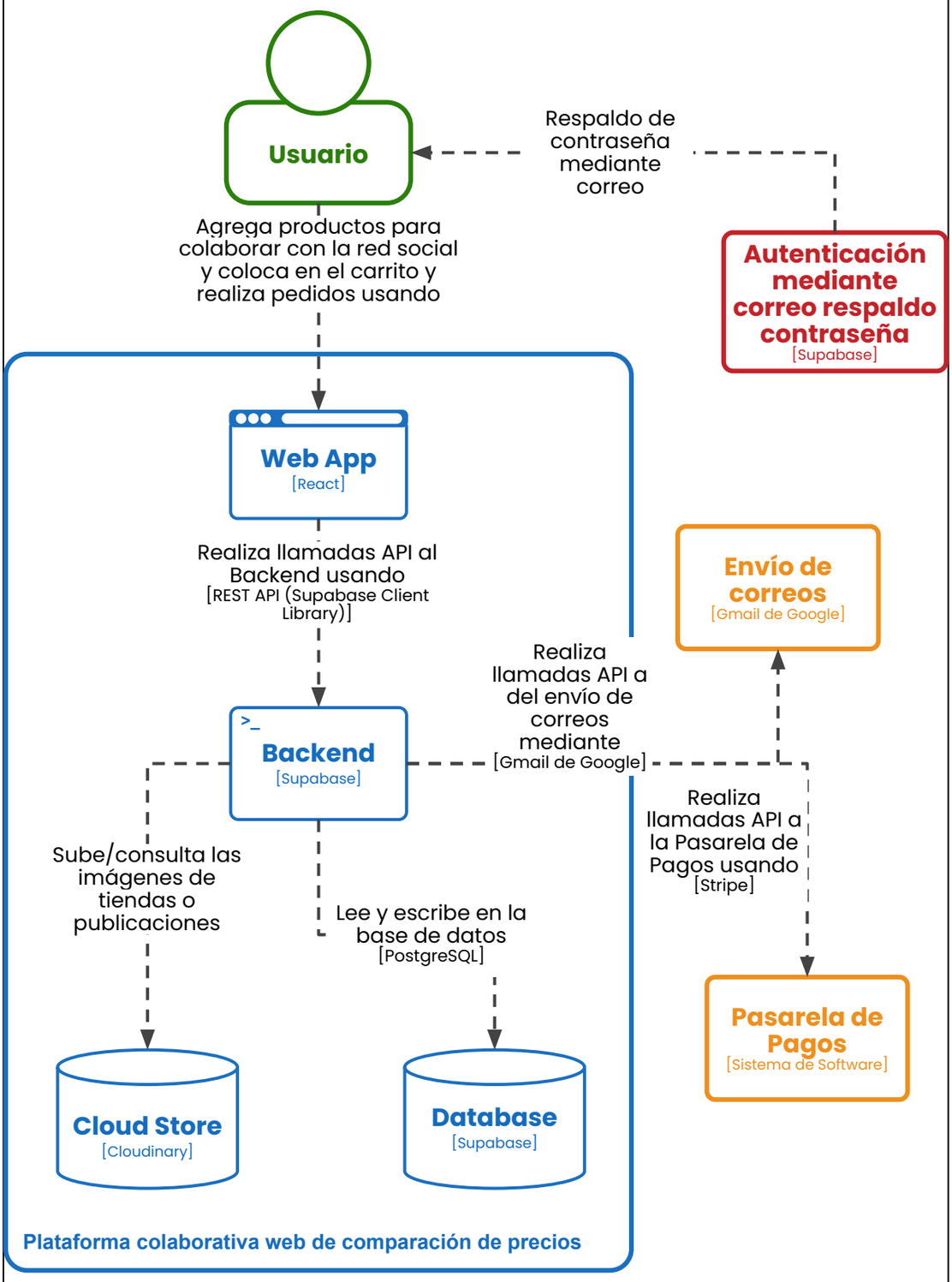


- **Moderador**

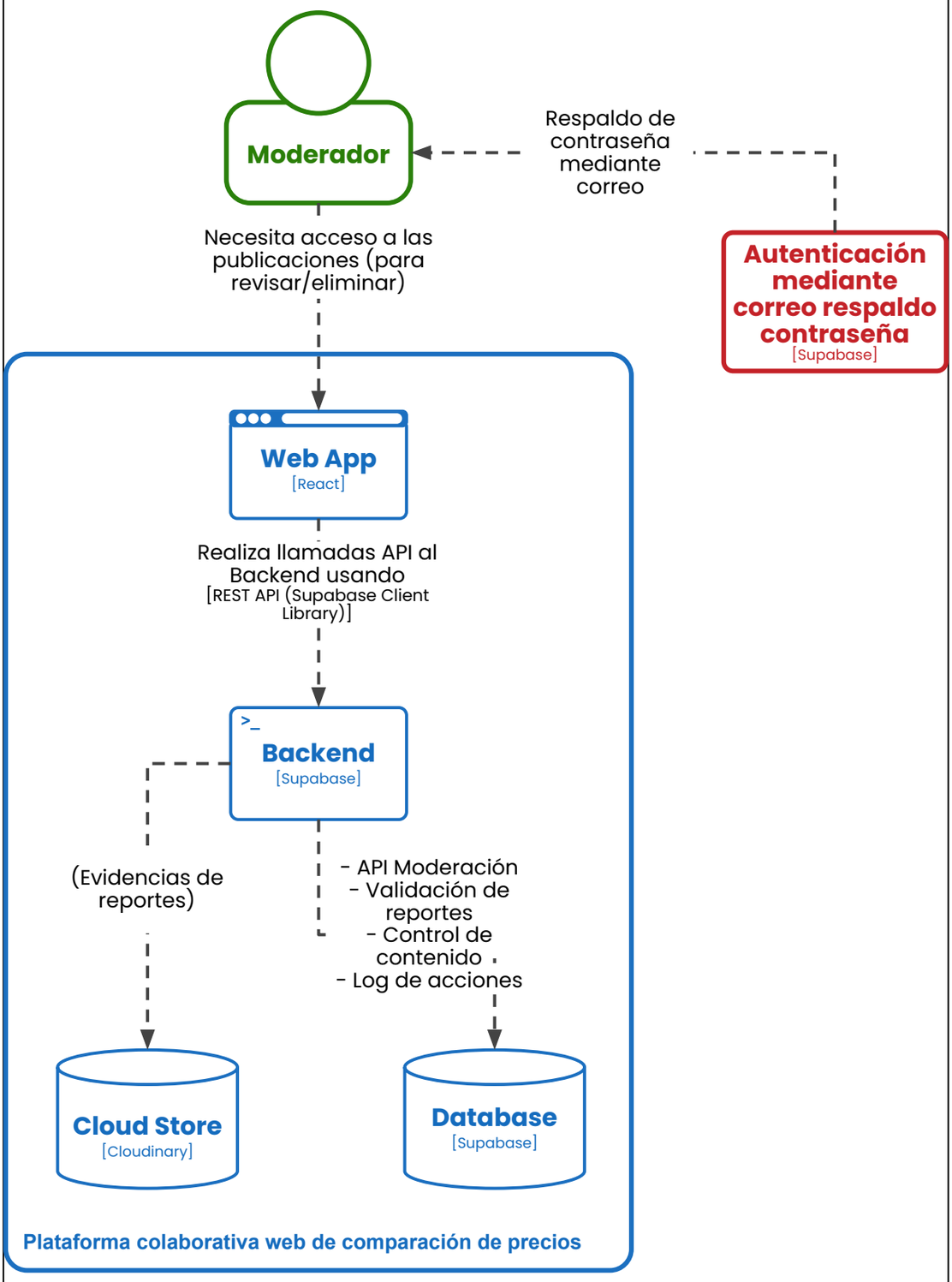


2.2 Diagrama de contenedores (N2)

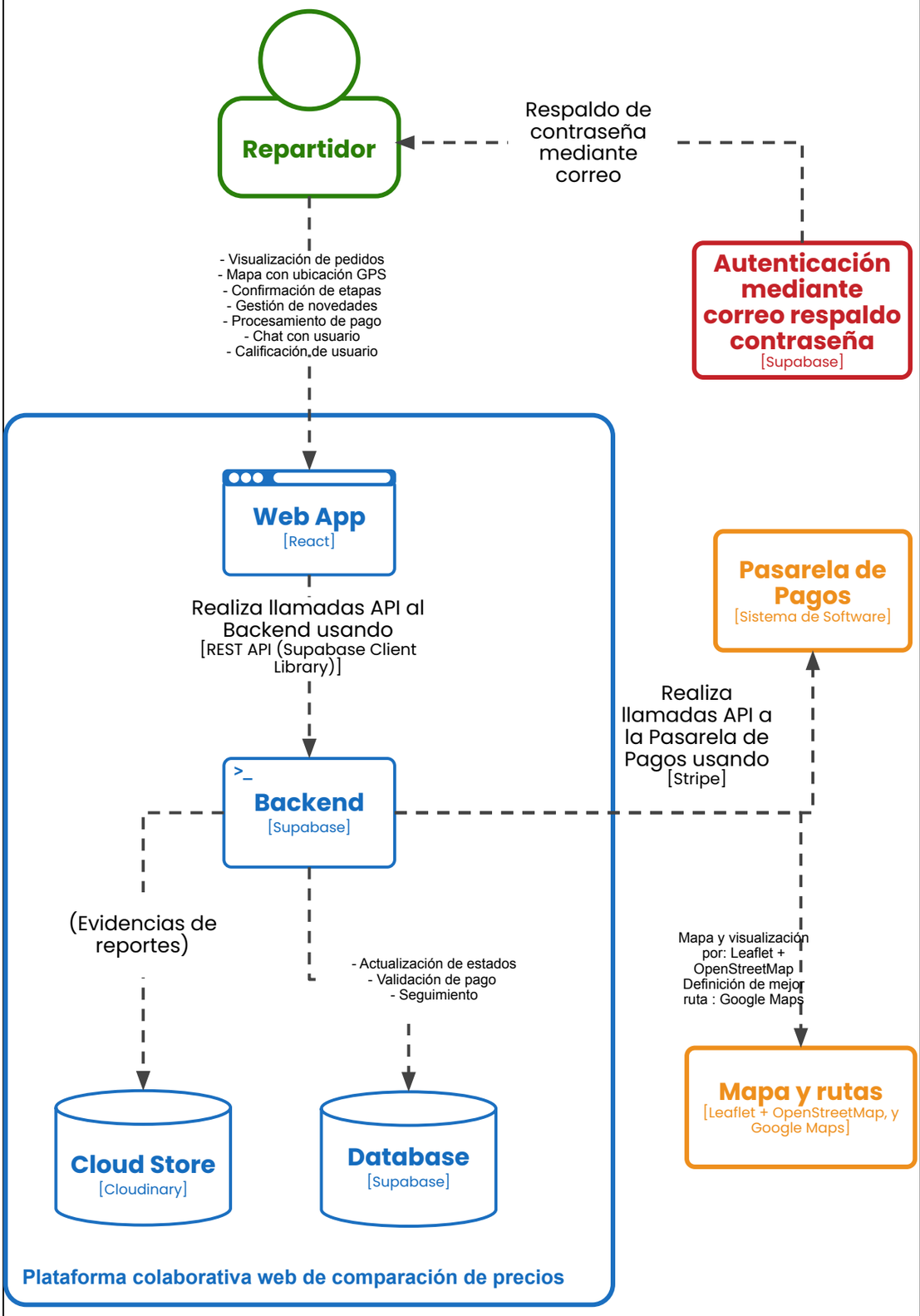
- **Usuario**



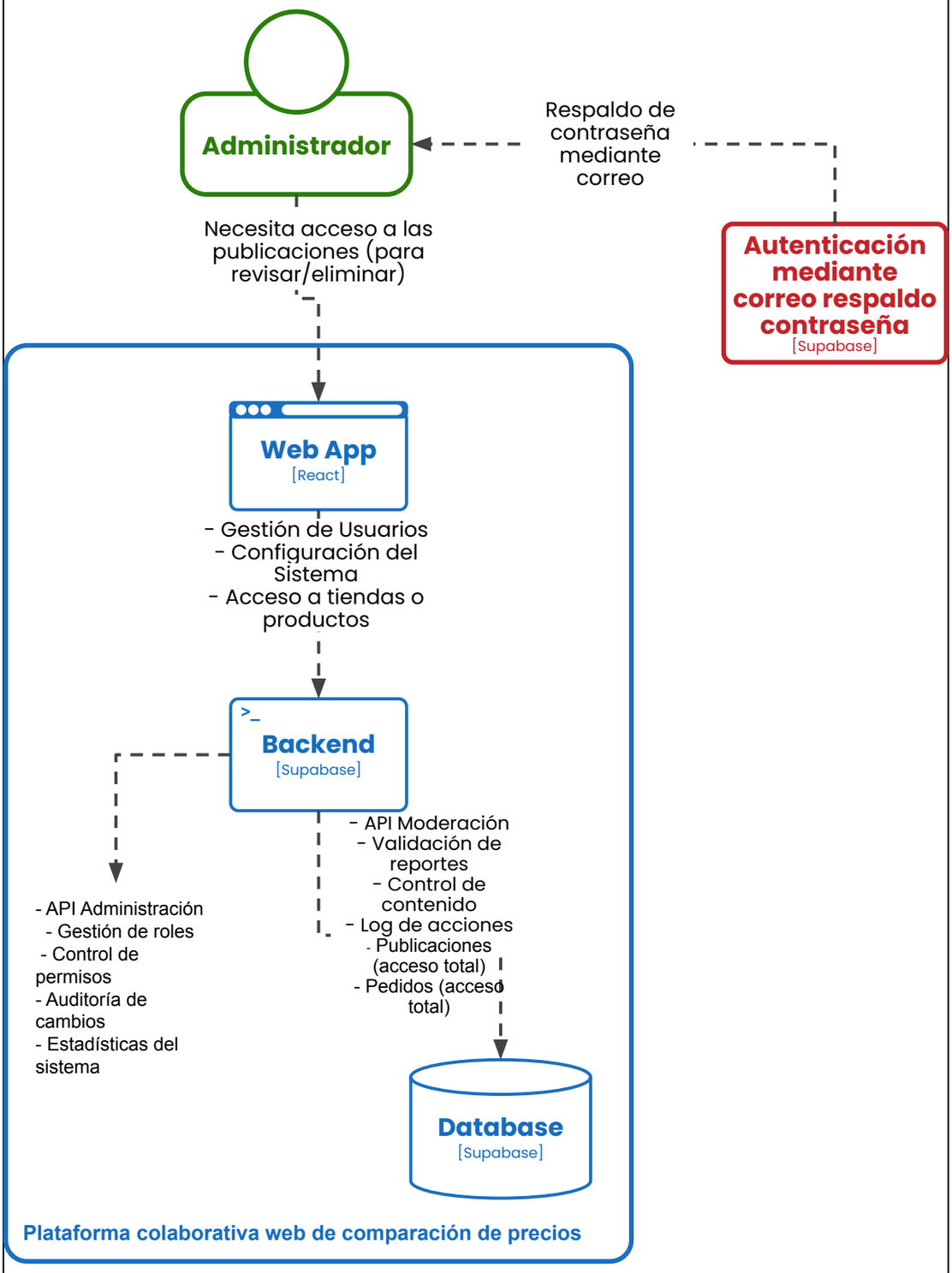
- **Moderador**


Moderador
Respaldo de contraseña mediante correo
Autenticación mediante correo respaldo contraseña
[Supabase]
Necesita acceso a las publicaciones (para revisar/eliminar)
Web App
[React]
Realiza llamadas API al Backend usando
[REST API (Supabase Client Library)]
Backend
[Supabase]
(Evidencias de reportes)
- API Moderación
- Validación de reportes
- Control de contenido
- Log de acciones
Cloud Store
[Cloudinary]
Database
[Supabase]
Plataforma colaborativa web de comparación de precios

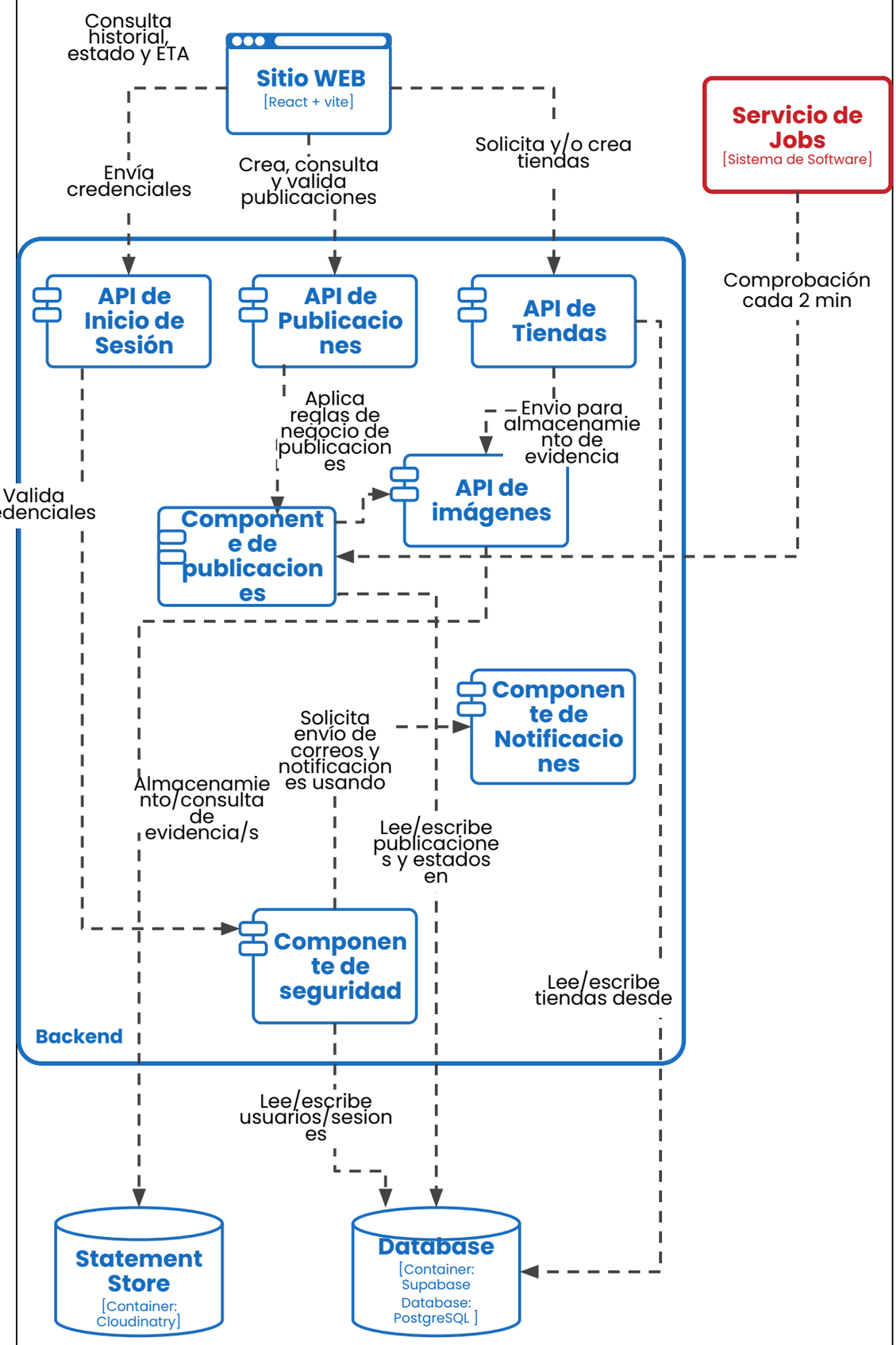
- **Repartidor**


Repartidor
Respaldo de contraseña mediante correo
Autenticación mediante correo respaldo contraseña
[Supabase]
- Visualización de pedidos
- Mapa con ubicación GPS
- Confirmación de etapas
- Gestión de novedades
- Procesamiento de pago
- Chat con usuario
- Calificación de usuario
Web App
[React]
Realiza llamadas API al Backend usando
[REST API (Supabase Client Library)]
Pasarela de Pagos
[Sistema de Software]
Realiza llamadas API a la Pasarela de Pagos usando
[Stripe]
Backend
[Supabase]
(Evidencias de reportes)
- Actualización de estados
- Validación de pago
- Seguimiento
Mapa y visualización por: Leaflet + OpenStreetMap
Definición de mejor ruta : Google Maps
Mapa y rutas
[Leaflet + OpenStreetMap, y Google Maps]
Cloud Store
[Cloudinary]
Database
[Supabase]
Plataforma colaborativa web de comparación de precios

- **Administrador**



2.3 Diagrama de componentes (N3)



2.4 Código (N4)

2.4.1 Estructura del repositorio y módulos

El repositorio de NØSEE está organizado bajo una arquitectura feature-based (orientada a módulos funcionales), donde cada capacidad del sistema agrupa de forma cohesiva sus propias páginas, componentes, hooks y schemas de validación. Esta decisión de diseño reduce el acoplamiento entre funcionalidades y permite que cada módulo evolucione de forma independiente sin afectar a los demás.

El código fuente se divide en tres grandes capas. La primera es la capa de presentación (src/features/, src/components/, src/pages/), que contiene toda la lógica de interfaz de usuario construida con React 19 y organizada por dominio funcional: autenticación, publicaciones de precios, tiendas, alertas y dashboards por rol. La segunda es la capa de servicios (src/services/), que centraliza toda la comunicación con los sistemas externos mediante un formato de respuesta uniforme { success, data/error }, desacoplando la UI de los detalles de integración con Supabase y Cloudinary. La tercera es la capa de infraestructura backend (supabase/), que contiene el esquema de base de datos PostgreSQL mediante migraciones versionadas y las Edge Functions de Deno encargadas de los procesos automatizados del sistema.

El control de acceso está implementado transversalmente mediante el componente ProtectedRoute.jsx y el store global authStore.js (Zustand), garantizando que cada ruta sea accesible únicamente por el rol autorizado. El enrutamiento del lado del cliente es gestionado por React Router DOM, con una regla de reescritura en vercel.json que asegura el correcto funcionamiento de la SPA en producción.

Estructura (árbol):

```
nosee/
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   │
│   ├── components/                 # Componentes globales reutilizables
│   │   ├── layout/
│   │   │   ├── Navbar.jsx
│   │   │   └── ProtectedRoute.jsx
│   │   └── ui/
│   │
│   ├── features/                   # Módulos funcionales (feature-based)
│   │   ├── auth/                   # Autenticación y perfiles de usuario
│   │   │   ├── pages/
│   │   │   ├── components/
│   │   │   ├── store/
│   │   │   │   └── authStore.js    # Estado global de sesión (Zustand)
│   │   │   ├── mappers/
│   │   │   └── schemas/
│   │   │
│   │   ├── publications/           # Publicaciones de precios
│   │   │   ├── pages/
│   │   │   ├── components/
│   │   │   ├── hooks/
│   │   │   └── schemas/
│   │   │
│   │   ├── stores/                 # Tiendas físicas y virtuales
│   │   │   ├── pages/
│   │   │   ├── components/
│   │   │   ├── hooks/
│   │   │   └── schemas/
│   │   │
│   │   ├── alerts/                 # Alertas de precio
│   │   │
│   │   └── dashboard/              # Dashboards por rol
│   │       ├── admin/
│   │       ├── moderator/
│   │       └── dealer/
│   │
│   ├── pages/                      # Páginas públicas generales
│   │   ├── HomePage.jsx
│   │   └── RankingPage.jsx
│   │
│   ├── router/                     # Enrutamiento y control de acceso
│   │   ├── App.routes.js
│   │   └── RoleRouter.jsx
│   │
│   ├── services/                   # Integración con servicios externos
│   │   ├── supabase.client.js
│   │   ├── cloudinary.js
│   │   └── api/
│   │       ├── auth.api.js
│   │       ├── users.api.js
│   │       ├── publications.api.js
│   │       ├── stores.api.js
│   │       └── alerts.api.js
│   │
│   ├── shared/                     # Utilidades transversales
│   │   └── utils/
│   │
├── supabase/                       # Infraestructura Backend
│   ├── migrations/                 # Esquema de base de datos
│   └── functions/                  # Edge Functions (Deno)
│       ├── notify-price-alerts/
│       └── verify-user/
│
├── tests/                          # Suite de pruebas
│   ├── unit/
│   ├── integration/
│   └── e2e/
│
├── vite.config.js
├── vercel.json
├── playwright.config.js
└── package.json
```

2.4 Código (N4)

2.4.2 Trazabilidad paquetes/Clases

Elemento N3	Código (paquete/clase)	Responsabilidad
Sitio WEB [React + Vite]	`src/App.jsx` `src/main.jsx` `src/router/App.routes.js` `src/router/RoleRouter.jsx`	Punto de entrada de la SPA. Define todas las rutas de la aplicación y redirige al usuario según su rol autenticado (Usuario, Moderador, Admin, Repartidor).
API de Inicio de Sesión	`src/services/api/auth.api.js` `src/services/supabase.client.js`	Expone las funciones de registro, login, logout, OAuth con Google, recuperación de contraseña y gestión del ciclo de vida de la sesión mediante Supabase Auth.
API de Publicaciones	`src/services/api/publications.api.js` `src/services/api/users.api.js`	CRUD completo de publicaciones de precios, sistema de upvote/downvote, reportes de contenido, búsqueda con filtros y paginación. Incluye timeouts adaptativos según visibilidad del navegador.
API de Tiendas	`src/services/api/stores.api.js`	CRUD de tiendas físicas y virtuales, búsqueda por proximidad geográfica con distancia Haversine, detección de tienda cercana con Nominatim y gestión de evidencias (máx. 3 por tienda).
API de Imágenes	`src/services/cloudinary.js`	Compresión del lado del cliente y subida de imágenes de evidencia a Cloudinary mediante API REST. Retorna la URL pública para su almacenamiento en la publicación o tienda.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-01: Canal del cliente mediante aplicación web responsive

- **Decisión:** Implementar el canal del cliente como una aplicación web responsive con React + vite.
- **Contexto:** El cliente requiere una experiencia móvil para realizar pedidos, consultar historial y recibir notificaciones, además de un soporte de web para tener más alcance y facilidad y velocidad de desarrollo..
- **Alternativas:** Aplicación web y aplicación móvil; app híbrida.
- **Justificación:** Permite una experiencia móvil como web de forma más sencilla y similar, controlando el ciclo de vida de la interfaz del cliente.
- **Consecuencias:** Se gestionan versiones por web, dificultad en adaptación en ambos dispositivos.

ADR-02: Arquitectura BaaS con Supabase

- **Decisión:** Desarrollar el proyecto mediante la arquitectura BaaS con Supabase.
- **Contexto:** Definir arquitectura del proyecto que permite concretar el proyecto de forma efectiva y eficaz.
- **Alternativas:** Arquitectura por capas, por microservicios, híbrida.
- **Justificación:** Permite un desarrollo rapida delegando responsabilidades al servicio.
- **Consecuencias:** Pérdida de control total sobre el backend, limitaciones del proveedor.

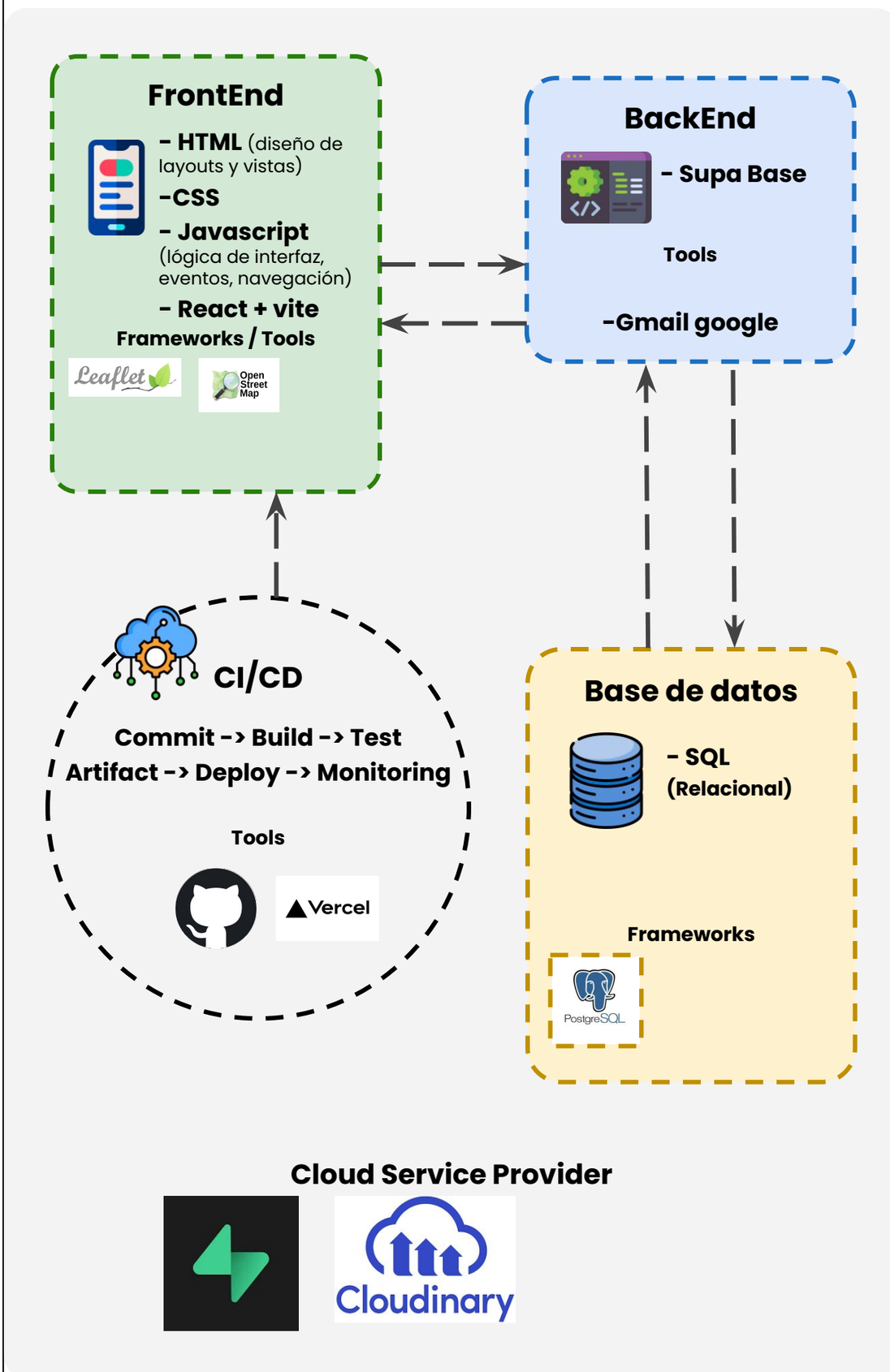
ADR-03: Servicio de guardado de imágenes con Cloudinary

- **Decisión:** Usar un servicio externo para el guardado de imágenes separado del backend con Cloudinary
- **Contexto:** Se requiere guardar imágenes de evide.
- **Alternativas:** Aplicación web responsive; app híbrida.
- **Justificación:** Permite una experiencia móvil nativa y control del ciclo de vida de la interfaz del cliente.
- **Consecuencias:** Se gestionan versiones y despliegues propios de Android.

3. DECISIONES DE ARQUITECTURA (ADRs)

3. DECISIONES DE ARQUITECTURA (ADRs)

4. STACK TECNOLÓGICO DEL SISTEMA



VIGILADA MINEDUCACIÓN